Universidade Federal Fluminense
LACOP, Laboratório de Comunicações Ópticas

Decifrando o código: previsão de consumo de energia

Explicação linha por linha para quem está aprendendo Python, com a leitura dos resultados obtidos

Antes de começar: o que este código faz

Este documento percorre, com calma e em detalhe, um código escrito em Python que prevê o consumo de energia de um prédio a partir de dados de sensores e de contexto. O texto foi pensado para quem está dando os primeiros passos em programação, então cada linha é explicada no seu próprio lugar, sem pressupor conhecimento prévio de bibliotecas de inteligência artificial.

O objetivo do código é um problema de regressão: prever o consumo de energia em quilowatt-hora, guardado na coluna `consumo_kwh`. Como o que se quer prever é um número contínuo, e não uma categoria, o modelo escolhido é a **Regressão Linear**, que busca a melhor relação matemática entre as informações de entrada e o valor a prever.

As informações de entrada misturam medições e contexto: a hora do dia, o dia da semana, se é fim de semana ou feriado, a temperatura e a umidade externas, a ocupação do prédio e se a tarifa está no horário de ponta. A ideia é que o consumo de energia dependa de todas essas condições ao mesmo tempo, e o modelo aprende como elas se combinam.

Bloco 1: as importações

Todo programa em Python começa trazendo as ferramentas que vai usar. Cada linha abaixo carrega uma peça específica.

```
import pandas as pd
```

Traz a biblioteca `pandas` com o apelido `pd`. É o pandas que abre o arquivo de dados e o organiza como uma tabela de linhas e colunas.

```
from sklearn.model_selection import train_test_split
```

Traz a função `train_test_split`, que separa os dados em uma parte para treino e outra para teste.

```
from sklearn.linear_model import LinearRegression
```

Traz o modelo `LinearRegression`, a Regressão Linear, que vai aprender a prever o consumo.

```
from sklearn.metrics import mean_absolute_error,
    mean_squared_error, r2_score
```

Traz três ferramentas de avaliação, todas próprias para regressão: `mean_absolute_error`, `mean_squared_error` e `r2_score`. Cada uma mede o desempenho de um jeito diferente, e as três serão interpretadas ao final.

```
from sklearn.preprocessing import OneHotEncoder
```

Traz o `OneHotEncoder`, que converte colunas de categorias, escritas como texto, em números que o modelo entende.

```
from sklearn.compose import ColumnTransformer
```

Traz o `ColumnTransformer`, que permite aplicar tratamentos diferentes a grupos diferentes de colunas de uma só vez.

```
from sklearn.pipeline import Pipeline
```

Traz o `Pipeline`, que encadeia a preparação dos dados e o modelo em uma sequência organizada.

Bloco 2: carregar o arquivo

```
df = pd.read_csv('consumo_energia_horario.csv')
```

Lê o arquivo de dados, no formato CSV, e guarda a tabela na variável `df`, abreviação de DataFrame, o nome que o pandas dá a uma tabela. Repare que aqui o nome do arquivo vem sem uma pasta na frente, o que significa que o programa vai procurá-lo na pasta atual de trabalho. No Google Colab, isso costuma funcionar quando o arquivo foi enviado para a pasta padrão.

Bloco 3: preparar os dados

Antes de o modelo aprender, os dados precisam estar no formato certo. Este bloco cuida disso, e cada decisão aqui influencia o resultado.

Definir o alvo

```
y = df['consumo_kwh']
```

Cria `y`, o alvo, contendo apenas a coluna `consumo_kwh`. Por convenção, o `y` minúsculo representa a resposta que o modelo deve prever.

Definir as entradas

```
X = df.drop(columns=['timestamp', 'consumo_kwh'])
```

Cria `X`, as entradas do modelo. Em vez de listar as colunas uma a uma, o código usa `drop`, que significa pegue todas as colunas, menos estas. Aqui ele remove duas: a coluna alvo `consumo_kwh`, que não pode ser entrada (seria entregar a resposta), e a coluna `timestamp`. O comentário do código explica por que o timestamp sai: a tabela já traz colunas de hora e dia da semana prontas, então a data e hora crua se tornou desnecessária.

Separar categorias de números

```
categorical_features = ['dia_semana']
```

Lista as colunas categóricas, as que guardam nomes em vez de valores numéricos. Aqui só a coluna `dia_semana`.

```
numerical_features = [
    'hora', 'fim_de_semana', 'feriado',
    'temperatura_externa_c', 'umidade_externa_pct',
    'ocupacao_predio', 'tarifa_ponta'
]
```

Lista as colunas numéricas, as que já são números. Essa separação é necessária porque, no passo seguinte, categorias e números recebem tratamentos diferentes.

Montar o preparador de dados

```
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'),
         categorical_features),
        ('num', 'passthrough', numerical_features)
    ])
```

Este é o coração da preparação. O `ColumnTransformer` aplica dois tratamentos, cada um ao seu grupo de colunas.

Ao grupo categórico, aplica o `OneHotEncoder`. Como o modelo só entende números, cada valor da coluna `dia_semana` vira uma coluna própria de zeros e uns. O parâmetro `handle_unknown='ignore'` evita que o programa quebre caso apareça, no teste, um dia da semana que não estava no treino.

Ao grupo numérico, aplica `'passthrough'`, que significa deixar passar sem mexer. Aqui está uma diferença importante em relação a outros projetos: as colunas numéricas não são normalizadas, apenas passam direto para o modelo. Essa é uma escolha válida, mas vale saber que, em alguns casos, normalizar os números poderia ajudar a Regressão Linear. Como os resultados foram excelentes, como veremos, o passthrough funcionou bem para esta base.

Bloco 4: dividir, treinar e prever

Dividir em treino e teste

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Separa os dados em quatro partes: entradas de treino e de teste, e alvos de treino e de teste. O `test_size=0.2` reserva vinte por cento das leituras para teste, dados que o modelo não vê durante o aprendizado e que servem para avaliá-lo com honestidade. O `random_state=42` faz a divisão ser sempre a mesma a cada execução, garantindo reprodutibilidade.

Montar e treinar o modelo

```
model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])
```

Cria um `Pipeline`, que encadeia o preparo dos dados e o modelo. Quando o código mandar treinar, os dados passam primeiro pelo `preprocessor` e só então chegam à `LinearRegression`. A vantagem é a consistência: o mesmo tratamento aplicado ao treino é aplicado automaticamente ao teste.

```
print("Treinando o modelo...")
model.fit(X_train, y_train)
print("Modelo treinado com sucesso!")
```

As mensagens de `print` apenas avisam na tela o que está acontecendo. A linha do meio, `model.fit`, é onde o aprendizado de fato ocorre: o modelo observa as entradas de treino e os consumos corretos, e ajusta seus parâmetros para relacionar uma coisa à outra. Depois dessa linha, o modelo está treinado.

Fazer as previsões

```
y_pred = model.predict(X_test)
```

Usa o modelo treinado para prever o consumo das leituras de teste, guardando os palpites em `y_pred`. São esses palpites que serão comparados com os consumos reais na avaliação.

Bloco 5: avaliar o modelo

```
print(f"Erro Médio Absoluto (MAE): ...")
print(f"Erro Quadrático Médio (MSE): ...")
print(f"R-quadrado (R2 Score): ...")
```

Estas três linhas calculam e imprimem as notas do modelo. Como este é um problema de regressão, as métricas medem o tamanho do erro e a qualidade do ajuste. Cada uma é explicada em detalhe na próxima seção, já com os valores obtidos.

Bloco 6: lendo os resultados obtidos

Com o código executado, a tela mostrou a mensagem de sucesso do treino e três métricas. Esta seção explica o que cada número significa e o conecta com as decisões do código.

Métrica	Valor obtido	Em uma frase
Erro Médio Absoluto (MAE)	2,06 kWh	Erro médio das previsões
Erro Quadrático Médio (MSE)	6,67	Penaliza erros grandes
R ao quadrado (R2)	0,98	Qualidade do ajuste

O MAE de 2,06

O erro médio absoluto diz de quanto o modelo erra, em média, na mesma unidade do que se prevê. Como o alvo é o consumo em quilowatt-hora, um MAE de 2,06 significa que, em média, a previsão do modelo fica a cerca de dois quilowatt-hora do consumo real. Se esse erro é pequeno ou grande depende da escala do consumo do prédio: para valores de consumo na casa das dezenas ou centenas de quilowatt-hora, um erro médio de dois é bastante baixo.

O MSE de 6,67

O erro quadrático médio também mede o erro, mas de um jeito diferente: antes de tirar a média, ele eleva cada erro ao quadrado. Isso faz com que erros grandes pesem muito mais do que erros pequenos. Serve como um alarme para prever se o modelo comete, de vez em quando, erros muito fora do normal. Um MSE de 6,67, próximo do quadrado do MAE, sugere que os erros são, em geral, parecidos entre si, sem muitos casos extremos. Se houvesse previsões muito erradas, o MSE seria bem maior que o quadrado do MAE.

O R2 de 0,98: um ajuste excelente

O R ao quadrado mede a proporção da variação do consumo que o modelo consegue explicar, numa escala de 0 a 1. Um **R2 de 0,98** é um resultado excelente: significa que o modelo explica noventa e oito por cento da variação do consumo de energia a partir das entradas que recebeu. Em outras palavras, as informações escolhidas, a hora, o dia, a temperatura, a ocupação, a tarifa e as demais, carregam quase toda a informação necessária para prever o consumo. Sobra muito pouco que o modelo não consiga explicar.

Correlacionando com o código

Vale notar como as escolhas do código levaram a esse resultado. A remoção da coluna timestamp fez sentido, porque as informações de tempo já estavam separadas em hora, dia da semana e indicadores de fim de semana e feriado, mais fáceis de o modelo usar. A conversão do dia da semana em colunas de zeros e uns permitiu que o modelo tratasse cada dia de forma independente. E a mistura de fatores físicos, como temperatura e umidade, com fatores de contexto, como feriado e tarifa de ponta, deu ao modelo um retrato completo das condições que afetam o consumo. O alto R2 indica que essa combinação de entradas foi muito bem escolhida.

Em resumo, o código cumpriu seu objetivo com folga. O modelo prevê o consumo de energia com erro médio baixo, sem sinais de erros extremos, e explica quase toda a variação dos dados. É um exemplo claro de como, quando as entradas certas são fornecidas, até um modelo simples como a Regressão Linear alcança um desempenho muito forte.
